МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний університет “Львівська політехніка”

Інститут комп’ютерних наук та інформаційних технологій

Кафедра автоматизованих систем управління

Звіт

про виконання лабораторної роботи №2

з дисципліни “ Паралельні обчислення і розподілені системи”

на тему:

“RESTful веб сервіси”

Виконав:

студент групи ОІ-21СП

Горбатюк Д. О.

Прийняла:

доцентка кафедри АСУ,

Федевич О. Ю.

Львів - 2025

Лабораторна робота №2

Тема: RESTful веб сервіси.

Мета роботи: Оволодіти практичними прийомами розробки RESTful веб сервісів.

Завдання лабораторної роботи

Варіант по списку групи - 1

Завдання: Написати RESTful веб сервіс, своєї предметної області: організація записів на квести, створення команд.

ХІД РОБОТИ

1. Мовою програмування обрано Java, адже ця мова програмування має доволі зручний фреймворк Spring Boot, для створення подібних клієнтів.
2. Створив проєкт у відповідному середовищі (див. Рис. 1.1):

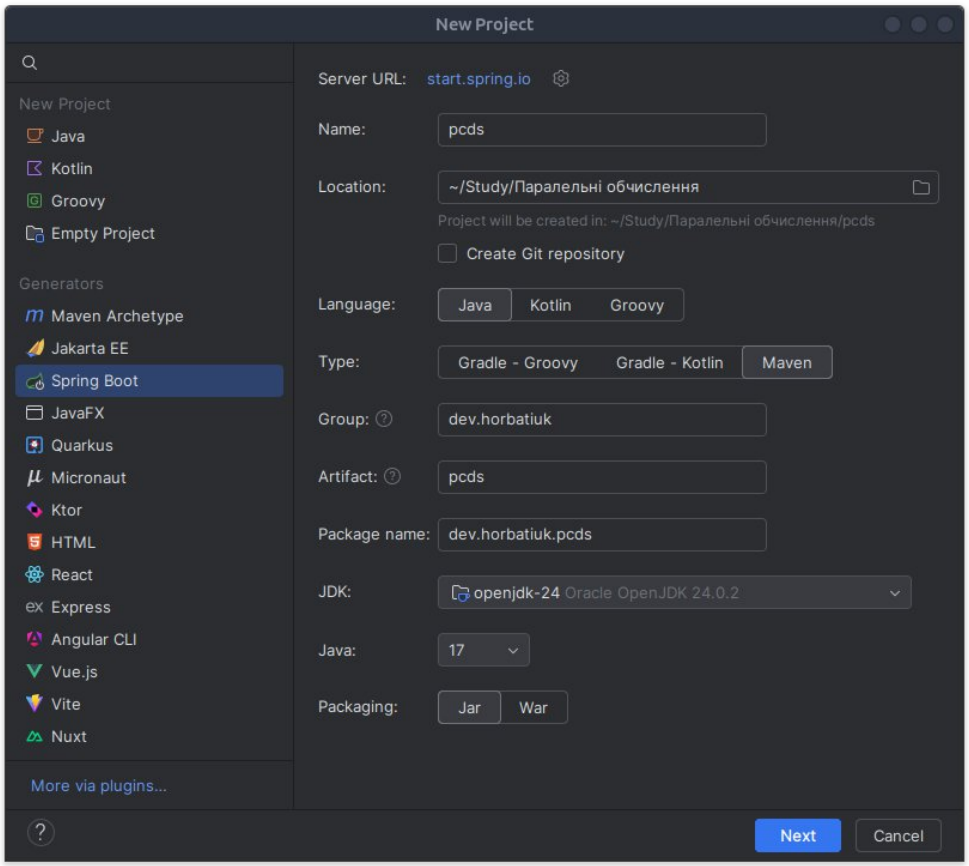
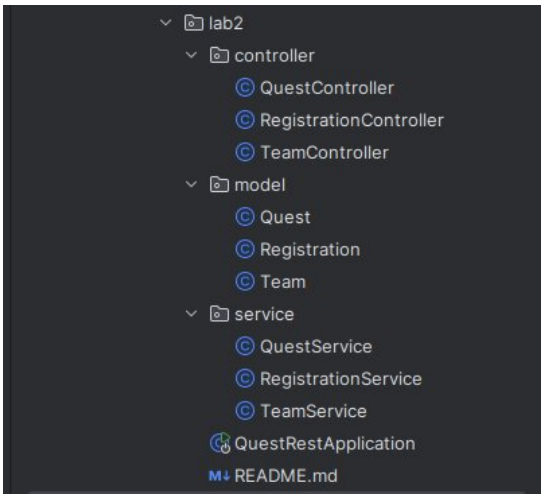


Рис. 1.1. Створення проєкту

3. Створив наступну архітектуру додатку:



4. Написав код всіх класів (див. Додатки 1-10).
5. Приклад виконання:

Блок 1 - приклад запиту curl, Блок 2 - відповідь на запит через консоль

```
echo "=== CREATE QUEST 1 ==="
curl -s -X POST http://localhost:8080/quests \
-H "Content-Type: application/json" \
-d '{"title":"Night City Quest","location":"Uzhhorod
Center","difficulty":"Medium","durationMinutes":90}' | jq

echo "=== CREATE QUEST 2 ==="
curl -s -X POST http://localhost:8080/quests \
-H "Content-Type: application/json" \
-d '{"title":"Castle Mystery","location":"Nevytske
Castle","difficulty":"Hard","durationMinutes":120}' | jq

echo "=== GET ALL QUESTS ==="
curl -s http://localhost:8080/quests | jq

echo "=== GET QUEST 1 ==="
curl -s http://localhost:8080/quests/1 | jq

echo "=== UPDATE QUEST 1 ==="
curl -s -X PUT http://localhost:8080/quests/1 \
-H "Content-Type: application/json" \
-d '{"title":"Night City Quest Updated","location":"Uzhhorod
Downtown","difficulty":"Hard","durationMinutes":100}' | jq

echo "=== GET UPDATED QUEST 1 ==="
curl -s http://localhost:8080/quests/1 | jq

echo "=== CREATE TEAM 1 ==="
curl -s -X POST http://localhost:8080/teams \
-H "Content-Type: application/json" \
```

```
-d '{"name":"Shadow Hunters","membersCount":4,"captainName":"Danylo"}' | jq

echo "=== CREATE TEAM 2 ==="
curl -s -X POST http://localhost:8080/teams \
-H "Content-Type: application/json" \
-d '{"name":"Code Raiders","membersCount":5,"captainName":"Andrii"}' | jq

echo "=== GET ALL TEAMS ==="
curl -s http://localhost:8080/teams | jq

echo "=== GET TEAM 1 ==="
curl -s http://localhost:8080/teams/1 | jq

echo "=== UPDATE TEAM 1 ==="
curl -s -X PUT http://localhost:8080/teams/1 \
-H "Content-Type: application/json" \
-d '{"name":"Shadow Hunters Elite","membersCount":6,"captainName":"Danylo
Horbatiuk"}' | jq

echo "=== GET UPDATED TEAM 1 ==="
curl -s http://localhost:8080/teams/1 | jq

echo "=== CREATE REGISTRATION 1 ==="
curl -s -X POST http://localhost:8080/registrations \
-H "Content-Type: application/json" \
-d '{"questId":1,"teamId":1,"date":"2026-03-29","status":"CONFIRMED"}' | jq

echo "=== CREATE REGISTRATION 2 ==="
curl -s -X POST http://localhost:8080/registrations \
-H "Content-Type: application/json" \
-d '{"questId":1,"teamId":1,"date":"2026-04-02","status":"PENDING"}' | jq

echo "=== GET ALL REGISTRATIONS ==="
curl -s http://localhost:8080/registrations | jq

echo "=== GET REGISTRATION 1 ==="
curl -s http://localhost:8080/registrations/1 | jq

echo "=== UPDATE REGISTRATION 1 ==="
curl -s -X PUT http://localhost:8080/registrations/1 \
-H "Content-Type: application/json" \
-d '{"questId":1,"teamId":1,"date":"2026-03-30","status":"COMPLETED"}' | jq

echo "=== GET UPDATED REGISTRATION 1 ==="
curl -s http://localhost:8080/registrations/1 | jq

echo "=== INVALID REGISTRATION: WRONG QUEST ID ==="
curl -s -X POST http://localhost:8080/registrations \
-H "Content-Type: application/json" \
-d '{"questId":99,"teamId":1,"date":"2026-03-29","status":"CONFIRMED"}' | jq -
R .
```

```
echo "=== INVALID REGISTRATION: WRONG TEAM ID ==="
curl -s -X POST http://localhost:8080/registrations \
-H "Content-Type: application/json" \
-d '{"questId":1,"teamId":99,"date":"2026-03-29","status":"CONFIRMED"}' | jq -
R .

echo "=== DELETE REGISTRATION 2 ==="
curl -s -X DELETE http://localhost:8080/registrations/2 -w "\nHTTP
STATUS: %{http_code}\n"

echo "=== GET ALL REGISTRATIONS AFTER DELETE ==="
curl -s http://localhost:8080/registrations | jq

echo "=== DELETE TEAM 2 ==="
curl -s -X DELETE http://localhost:8080/teams/2 -w "\nHTTP
STATUS: %{http_code}\n"

echo "=== GET ALL TEAMS AFTER DELETE ==="
curl -s http://localhost:8080/teams | jq

echo "=== DELETE QUEST 2 ==="
curl -s -X DELETE http://localhost:8080/quests/2 -w "\nHTTP
STATUS: %{http_code}\n"

echo "=== GET ALL QUESTS AFTER DELETE ==="
curl -s http://localhost:8080/quests | jq
```

```
=== CREATE QUEST 1 ===
{
  "id": 1,
  "title": "Night City Quest",
  "location": "Uzhhorod Center",
  "difficulty": "Medium",
  "durationMinutes": 90
}
=== CREATE QUEST 2 ===
{
  "id": 2,
  "title": "Castle Mystery",
  "location": "Nevytske Castle",
  "difficulty": "Hard",
  "durationMinutes": 120
}
=== GET ALL QUESTS ===
[
  {
    "id": 1,
    "title": "Night City Quest",
    "location": "Uzhhorod Center",
    "difficulty": "Medium",
    "durationMinutes": 90
```

```
  },
  {
    "id": 2,
    "title": "Castle Mystery",
    "location": "Nevytske Castle",
    "difficulty": "Hard",
    "durationMinutes": 120
  }
]
=== GET QUEST 1 ===
{
  "id": 1,
  "title": "Night City Quest",
  "location": "Uzhhorod Center",
  "difficulty": "Medium",
  "durationMinutes": 90
}
=== UPDATE QUEST 1 ===
{
  "id": 1,
  "title": "Night City Quest Updated",
  "location": "Uzhhorod Downtown",
  "difficulty": "Hard",
  "durationMinutes": 100
}
=== GET UPDATED QUEST 1 ===
{
  "id": 1,
  "title": "Night City Quest Updated",
  "location": "Uzhhorod Downtown",
  "difficulty": "Hard",
  "durationMinutes": 100
}
=== CREATE TEAM 1 ===
{
  "id": 1,
  "name": "Shadow Hunters",
  "membersCount": 4,
  "captainName": "Danylo"
}
=== CREATE TEAM 2 ===
{
  "id": 2,
  "name": "Code Raiders",
  "membersCount": 5,
  "captainName": "Andrii"
}
=== GET ALL TEAMS ===
[
  {
    "id": 1,
    "name": "Shadow Hunters",
    "membersCount": 4,
    "captainName": "Danylo"
```

```
  },
  {
    "id": 2,
    "name": "Code Raiders",
    "membersCount": 5,
    "captainName": "Andrii"
  }
]
=== GET TEAM 1 ===
{
  "id": 1,
  "name": "Shadow Hunters",
  "membersCount": 4,
  "captainName": "Danylo"
}
=== UPDATE TEAM 1 ===
{
  "id": 1,
  "name": "Shadow Hunters Elite",
  "membersCount": 6,
  "captainName": "Danylo Horbatiuk"
}
=== GET UPDATED TEAM 1 ===
{
  "id": 1,
  "name": "Shadow Hunters Elite",
  "membersCount": 6,
  "captainName": "Danylo Horbatiuk"
}
=== CREATE REGISTRATION 1 ===
{
  "id": 1,
  "questId": 1,
  "teamId": 1,
  "date": "2026-03-29",
  "status": "CONFIRMED"
}
=== CREATE REGISTRATION 2 ===
{
  "id": 2,
  "questId": 1,
  "teamId": 1,
  "date": "2026-04-02",
  "status": "PENDING"
}
=== GET ALL REGISTRATIONS ===
[
  {
    "id": 1,
    "questId": 1,
    "teamId": 1,
    "date": "2026-03-29",
    "status": "CONFIRMED"
  },
```

```
  {
    "id": 2,
    "questId": 1,
    "teamId": 1,
    "date": "2026-04-02",
    "status": "PENDING"
  }
]
=== GET REGISTRATION 1 ===
{
  "id": 1,
  "questId": 1,
  "teamId": 1,
  "date": "2026-03-29",
  "status": "CONFIRMED"
}
=== UPDATE REGISTRATION 1 ===
{
  "id": 1,
  "questId": 1,
  "teamId": 1,
  "date": "2026-03-30",
  "status": "COMPLETED"
}
=== GET UPDATED REGISTRATION 1 ===
{
  "id": 1,
  "questId": 1,
  "teamId": 1,
  "date": "2026-03-30",
  "status": "COMPLETED"
}
=== INVALID REGISTRATION: WRONG QUEST ID ===
"Quest with given id does not exist"
=== INVALID REGISTRATION: WRONG TEAM ID ===
"Team with given id does not exist"
=== DELETE REGISTRATION 2 ===

HTTP STATUS: 204
=== GET ALL REGISTRATIONS AFTER DELETE ===
[
  {
    "id": 1,
    "questId": 1,
    "teamId": 1,
    "date": "2026-03-30",
    "status": "COMPLETED"
  }
]
=== DELETE TEAM 2 ===

HTTP STATUS: 204
=== GET ALL TEAMS AFTER DELETE ===
[
```

```
  {
    "id": 1,
    "name": "Shadow Hunters Elite",
    "membersCount": 6,
    "captainName": "Danylo Horbatiuk"
  }
]
=== DELETE QUEST 2 ===

HTTP STATUS: 204
=== GET ALL QUESTS AFTER DELETE ===
[
  {
    "id": 1,
    "title": "Night City Quest Updated",
    "location": "Uzhhorod Downtown",
    "difficulty": "Hard",
    "durationMinutes": 100
  }
]
```

6. REST-клієнт передбачає наступні ендпоінти:

REST endpoints

Quests

GET /quests — отримати всі квести

GET /quests/{id} — отримати квест за id

POST /quests — створити квест

PUT /quests/{id} — оновити квест

DELETE /quests/{id} — видалити квест

Teams

GET /teams — отримати всі команди

GET /teams/{id} — отримати команду за id

POST /teams — створити команду

PUT /teams/{id} — оновити команду

DELETE /teams/{id} — видалити команду

Registrations

GET /registrations — отримати всі реєстрації

GET /registrations/{id} — отримати реєстрацію за id

POST /registrations — створити реєстрацію

PUT /registrations/{id} — оновити реєстрацію

DELETE /registrations/{id} — видалити реєстрацію

ВИСНОВКИ

У ході лабораторної роботи я реалізував REST-клієнт мовою Java за обраною доменною областю: організація заходів, створення команд і квестів. Перелік створених ендпоінтів наведений вище.

Тестування ендпоінтів відбувалось за допомогою curl і термінал, що є доволі швидким рішенням, для тестування, яке не вимагає встановлення додаткового ПЗ.

Ця лабораторна робота справді показала, що RESTful сервіси виступають базовим способом взаємодії між окремими компонентами розподілених систем. Саме через подібні API різні модулі, сервери або мікросервіси можуть обмінюватися даними між собою незалежно від платформи чи мови реалізації.

ДОДАТКИ

Додаток 1

Код файлу QuestController.java

```
package dev.horbatiuk.pcds.lab2.controller;

import dev.horbatiuk.pcds.lab2.model.Quest;
import dev.horbatiuk.pcds.lab2.service.QuestService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/quests")
public class QuestController {

    private final QuestService questService;

    public QuestController(QuestService questService) {
        this.questService = questService;
    }

    @GetMapping
    public List<Quest> getAll() {
        return questService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Quest> getById(@PathVariable Long id) {
        Quest quest = questService.getById(id);
        return quest != null ? ResponseEntity.ok(quest) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Quest> create(@RequestBody Quest quest) {
        return ResponseEntity.ok(questService.create(quest));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Quest> update(@PathVariable Long id, @RequestBody Quest
quest) {
        Quest updated = questService.update(id, quest);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
```

```
        return questService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 2

Код файлу RegistrationController.java

```
package dev.horbatiuk.pcds.lab2.controller;

import dev.horbatiuk.pcds.lab2.model.Registration;
import dev.horbatiuk.pcds.lab2.service.QuestService;
import dev.horbatiuk.pcds.lab2.service.RegistrationService;
import dev.horbatiuk.pcds.lab2.service.TeamService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/registrations")
public class RegistrationController {

    private final RegistrationService registrationService;
    private final QuestService questService;
    private final TeamService teamService;

    public RegistrationController(RegistrationService registrationService,
                                  QuestService questService,
                                  TeamService teamService) {
        this.registrationService = registrationService;
        this.questService = questService;
        this.teamService = teamService;
    }

    @GetMapping
    public List<Registration> getAll() {
        return registrationService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Registration> getById(@PathVariable Long id) {
        Registration registration = registrationService.getById(id);
        return registration != null ? ResponseEntity.ok(registration) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody Registration registration) {
        if (!questService.exists(registration.getQuestId())) {
            return ResponseEntity.badRequest().body("Quest with given id does not
exist");
        }
        if (!teamService.exists(registration.getTeamId())) {
            return ResponseEntity.badRequest().body("Team with given id does not
exist");
```

```
        }
        return ResponseEntity.ok(registrationService.create(registration));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody
Registration registration) {
        if (!questService.exists(registration.getQuestId())) {
            return ResponseEntity.badRequest().body("Quest with given id does not
exist");
        }
        if (!teamService.exists(registration.getTeamId())) {
            return ResponseEntity.badRequest().body("Team with given id does not
exist");
        }

        Registration updated = registrationService.update(id, registration);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return registrationService.delete(id) ?
ResponseEntity.noContent().build() : ResponseEntity.notFound().build();
    }
}
```

Додаток 3

Код файлу TeamController.java

```
package dev.horbatiuk.pcds.lab2.controller;

import dev.horbatiuk.pcds.lab2.model.Team;
import dev.horbatiuk.pcds.lab2.service.TeamService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/teams")
public class TeamController {

    private final TeamService teamService;

    public TeamController(TeamService teamService) {
        this.teamService = teamService;
    }

    @GetMapping
    public List<Team> getAll() {
        return teamService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Team> getById(@PathVariable Long id) {
        Team team = teamService.getById(id);
        return team != null ? ResponseEntity.ok(team) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Team> create(@RequestBody Team team) {
        return ResponseEntity.ok(teamService.create(team));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Team> update(@PathVariable Long id, @RequestBody Team
team) {
        Team updated = teamService.update(id, team);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return teamService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
```

```
        }
}
```

Додаток 4

Код файлу Quest.java

```
package dev.horbatiuk.pcds.lab2.model;

public class Quest {
    private Long id;
    private String title;
    private String location;
    private String difficulty;
    private int durationMinutes;

    public Quest() {
    }

    public Quest(Long id, String title, String location, String difficulty, int
durationMinutes) {
        this.id = id;
        this.title = title;
        this.location = location;
        this.difficulty = difficulty;
        this.durationMinutes = durationMinutes;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getLocation() {
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public String getDifficulty() {
        return difficulty;
    }
```

```
    public void setDifficulty(String difficulty) {
        this.difficulty = difficulty;
    }

    public int getDurationMinutes() {
        return durationMinutes;
    }

    public void setDurationMinutes(int durationMinutes) {
        this.durationMinutes = durationMinutes;
    }
}
```

Додаток 5

Код файлу Registration.java

```
package dev.horbatiuk.pcds.lab2.model;

public class Registration {
    private Long id;
    private Long questId;
    private Long teamId;
    private String date;
    private String status;

    public Registration() {
    }

    public Registration(Long id, Long questId, Long teamId, String date, String
status) {
        this.id = id;
        this.questId = questId;
        this.teamId = teamId;
        this.date = date;
        this.status = status;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public Long getQuestId() {
        return questId;
    }

    public void setQuestId(Long questId) {
        this.questId = questId;
    }

    public Long getTeamId() {
        return teamId;
    }

    public void setTeamId(Long teamId) {
        this.teamId = teamId;
    }

    public String getDate() {
        return date;
    }
```

```
    public void setDate(String date) {
        this.date = date;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

Додаток 6

Код файлу Team.java

```
package dev.horbatiuk.pcds.lab2.model;

public class Team {
    private Long id;
    private String name;
    private int membersCount;
    private String captainName;

    public Team() {
    }

    public Team(Long id, String name, int membersCount, String captainName) {
        this.id = id;
        this.name = name;
        this.membersCount = membersCount;
        this.captainName = captainName;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getMembersCount() {
        return membersCount;
    }

    public void setMembersCount(int membersCount) {
        this.membersCount = membersCount;
    }

    public String getCaptainName() {
        return captainName;
    }

    public void setCaptainName(String captainName) {
```

```
        this.captainName = captainName;
    }
}
```

Додаток 7

Код файлу QuestService.java

```
package dev.horbatiuk.pcds.lab2.service;

import dev.horbatiuk.pcds.lab2.model.Quest;
import org.springframework.stereotype.Service;

import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class QuestService {
    private final Map<Long, Quest> quests = new HashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Quest> getAll() {
        return new ArrayList<>(quests.values());
    }

    public Quest getById(Long id) {
        return quests.get(id);
    }

    public Quest create(Quest quest) {
        Long id = counter.getAndIncrement();
        quest.setId(id);
        quests.put(id, quest);
        return quest;
    }

    public Quest update(Long id, Quest updatedQuest) {
        if (!quests.containsKey(id)) {
            return null;
        }
        updatedQuest.setId(id);
        quests.put(id, updatedQuest);
        return updatedQuest;
    }

    public boolean delete(Long id) {
        return quests.remove(id) != null;
    }

    public boolean exists(Long id) {
        return quests.containsKey(id);
    }
}
```

Додаток 8

Код файлу RegistrationService.java

```
package dev.horbatiuk.pcds.lab2.service;

import dev.horbatiuk.pcds.lab2.model.Registration;
import org.springframework.stereotype.Service;

import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class RegistrationService {
    private final Map<Long, Registration> registrations = new HashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Registration> getAll() {
        return new ArrayList<>(registrations.values());
    }

    public Registration getById(Long id) {
        return registrations.get(id);
    }

    public Registration create(Registration registration) {
        Long id = counter.getAndIncrement();
        registration.setId(id);
        registrations.put(id, registration);
        return registration;
    }

    public Registration update(Long id, Registration updatedRegistration) {
        if (!registrations.containsKey(id)) {
            return null;
        }
        updatedRegistration.setId(id);
        registrations.put(id, updatedRegistration);
        return updatedRegistration;
    }

    public boolean delete(Long id) {
        return registrations.remove(id) != null;
    }
}
```

Додаток 9

Код файлу TeamService.java

```
package dev.horbatiuk.pcds.lab2.service;

import dev.horbatiuk.pcds.lab2.model.Team;
import org.springframework.stereotype.Service;

import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class TeamService {
    private final Map<Long, Team> teams = new HashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Team> getAll() {
        return new ArrayList<>(teams.values());
    }

    public Team getById(Long id) {
        return teams.get(id);
    }

    public Team create(Team team) {
        Long id = counter.getAndIncrement();
        team.setId(id);
        teams.put(id, team);
        return team;
    }

    public Team update(Long id, Team updatedTeam) {
        if (!teams.containsKey(id)) {
            return null;
        }
        updatedTeam.setId(id);
        teams.put(id, updatedTeam);
        return updatedTeam;
    }

    public boolean delete(Long id) {
        return teams.remove(id) != null;
    }

    public boolean exists(Long id) {
        return teams.containsKey(id);
    }
}
```

Додаток 10

Код файлу QuestRestApplication.java

```
package dev.horbatiuk.pcds.lab2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class QuestRestApplication {
    public static void main(String[] args) {
        SpringApplication.run(QuestRestApplication.class, args);
    }
}
```